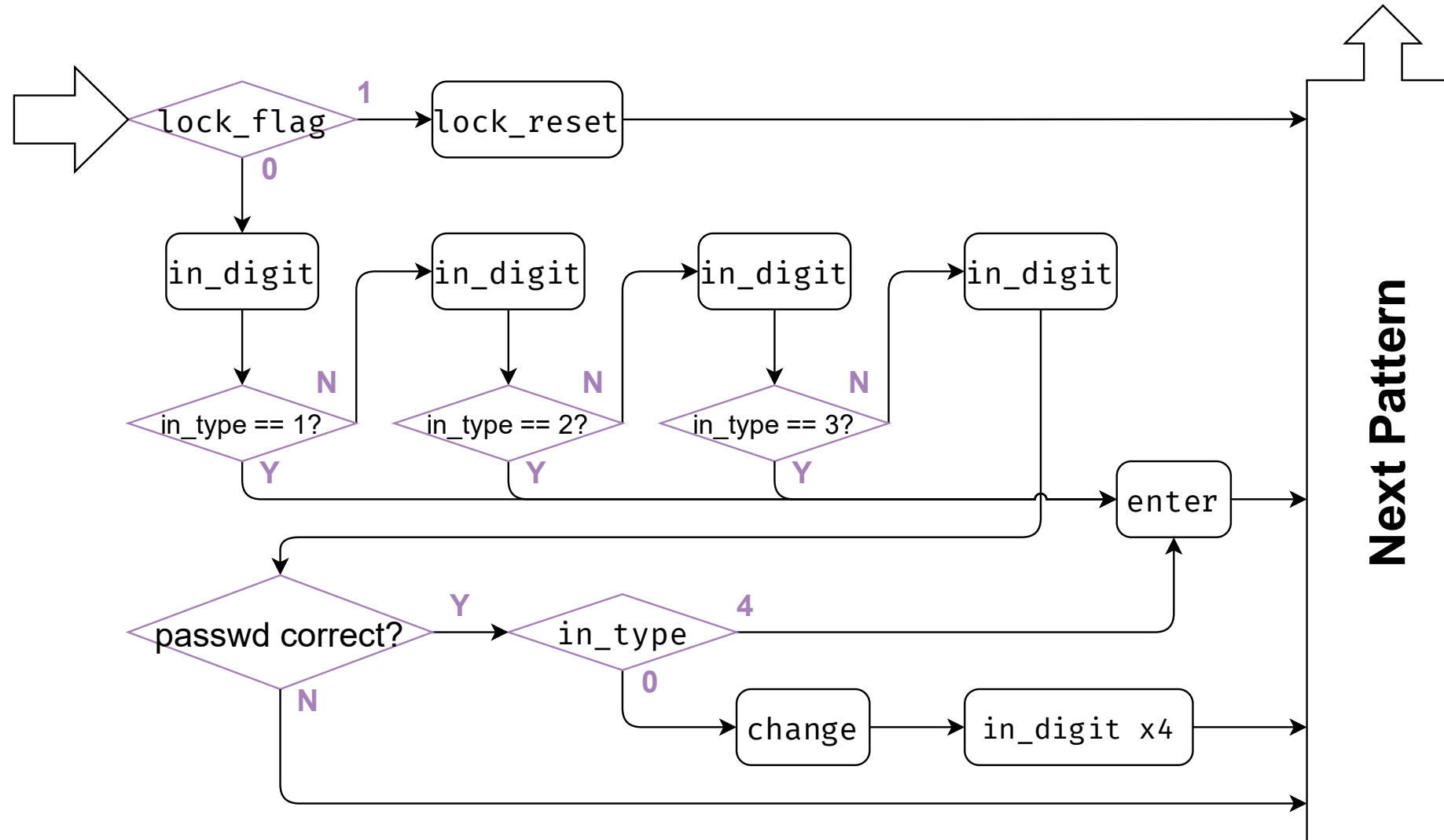




Lab06 Code Review

林群凱

Flowchart



Clock & Reset

Toggle every half cycle time →

```
real CYCLE = `CYCLE_TIME;
always #(CYCLE/2.0) clk = ~clk;
```

- Keep clock unchanged during reset
- Do not check immediately after reset

```
task reset_task();
    // Default values for design inputs
    rst_n = 1'b1;
    ...

    force clk = 1'b0;
    #(CYCLE * ($random() % 'd3 + 'd1)); rst_n = 1'b0;
    #(CYCLE * 0.75);

    if (state !== LOCKED || unlocked !== 1'b0 || alarm !== 1'b0) begin : RESET_CHECK
        fail(0);
        #(CYCLE * 5);
        $fclose(f_in);
        $finish;
    end

    #(CYCLE * 0.25); rst_n = 'b1;
    #(CYCLE); release clk;
    repeat(5) @(negedge clk);
endtask
```

Main Flow

- Software-like sequential flow + timing control
- Wrap the related code snippets into task blocks

```
initial begin
    f_in = $fopen("../00_TESTBED/input.txt", "r");
    status = $fscanf(f_in, "%d", pat_num);
    reset_task();
    for(pat_cnt = 0; pat_cnt < pat_num; pat_cnt++) begin
        read_file();
        input_task();
    end
    $fclose(f_in);
    YOU_PASS_task();
end
```

Reading File

- C-style function
- Read into variables in multiple \$fscanf calls
- Some goldens can be determined in advance



```
task read_file();
    status = $fscanf(f_in, "%d", lock_flag);
    if (!lock_flag) begin
        status = $fscanf(f_in, "%d %d %d %d %d", in_d[0], in_d[1], in_d[2], in_d[3], in_type);
        if (in_type == 0) begin
            status = $fscanf(f_in, "%d %d %d %d", ch_d[0], ch_d[1], ch_d[2], ch_d[3]);
        end
    end
    golden_unlocked = (in_d == golden_passwd);
endtask
```

Reference Code — lock_reset

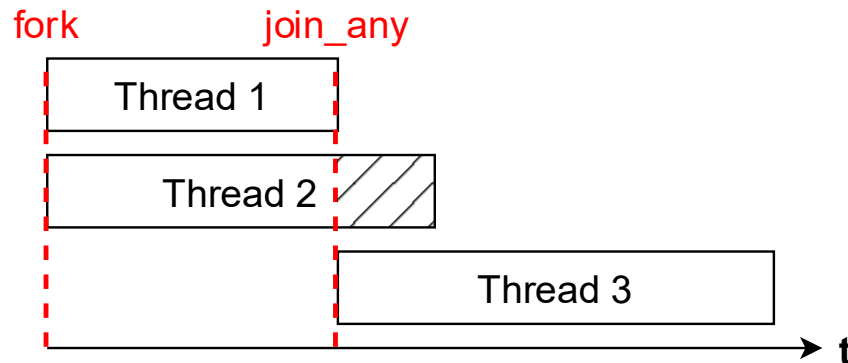
- Reset status and password

```
task input_task();  
    if (lock_flag) begin  
        in_valid = 1'b1;  
        lock_reset = 1'b1;  
        @(negedge clk);  
        in_valid = 1'b0;  
        lock_reset = 1'b0;  
  
        attempt = 0;  
        golden_alarm = 1'b0;  
        golden_passwd = '{2, 0, 2, 6};  
        repeat($random() % 'd6 + 'd5) @(negedge clk);  
  
    end else begin  
        ...  
    end  
end
```

Reference Code — CHECK 4

- Continuously check while giving inputs
- Use fork-join_any to capture the thread that finishes first

Never satisfied if not in ALARM



```
fork
begin
    key_in_task();
    repeat($random() % 'd6 + 'd5) @ (negedge clk);
end
begin : ALARM_DETECTION
    wait(golden_alarm);
    wait(alarm !== 1'b1 || state !== ALARM);
    fail(4);
    #(CYCLE * 5);
    $fclose(f_in);
    $finish;
end
join_any
disable fork;
```

Reference Code — CHECK 4 (Another Approach)

- Continuously check while giving inputs
- Use a parallel forever loop
 - Start check after reset
 - Delay a small interval to avoid race condition

```
initial begin
    wait(rst_n === 1'b0);
    wait(rst_n === 1'b1);
    forever begin
        @(negedge clk); #(CYCLE * 0.1);
        if(golden_alarm && (alarm !== 1'b1 || state !== ALARM)) begin
            fail(4);
            #(CYCLE * 5);
            $fclose(f_in);
            $finish;
        end
    end
end
```


Reference Code — CHECK 1

- Use nested if-else condition or early exit when sending enter

Only check when not in ALARM →

Early exit the task →

```
task key_in_task();  
    // 1st digit  
    in_valid = 1'b1; in_digit = in_d[0];  
    @(negedge clk);  
    in_valid = 1'b0; in_digit = 'dx;  
    repeat($random() % 'd3) @(negedge clk);  
    if (in_type == 1) begin  
        in_valid = 1'b1; enter = 1'b1;  
        @(negedge clk);  
        if (!golden_alarm && state != LOCKED) begin  
            fail(1);  
            #(CYCLE * 5);  
            $fclose(f_in);  
            $finish;  
        end  
        in_valid = 1'b0; enter = 1'b0;  
        return;  
    end  
    ...  
end
```

Reference Code — CHECK 2 & 3

Only check when not in ALARM →

Record the golden status →

```
// 4th digit
in_valid = 1'b1; in_digit = in_d[3];
@(negedge clk);
in_valid = 1'b0; in_digit = 'dx;
@(negedge clk);
// Verify
attempt = golden_unlocked ? 0 : attempt + 1;
if (!golden_alarm) begin
    if (golden_unlocked && (state !== UNLOCKED || unlocked !== 1'b1)) begin
        fail(2);
        #(CYCLE * 5);
        $fclose(f_in);
        $finish;
    end if (!golden_unlocked) begin
        if (attempt == 3) begin
            golden_alarm = 1'b1;
        end else begin
            if (state !== LOCKED) begin
                fail(3);
                #(CYCLE * 5);
                $fclose(f_in);
                $finish;
            end
        end
    end
end
end
...
```

Reference Code — Change Password

- Only have actions when unlocked

```
// Correct password
if (golden_unlocked && in_type == 4) begin
    in_valid = 1'b1; enter = 1'b1;
    @(negedge clk);
    in_valid = 1'b0; enter = 1'b0;

// Change password
end else if (golden_unlocked && in_type == 0) begin
    in_valid = 1'b1; change = 1'b1;
    @(negedge clk);
    in_valid = 1'b0; change = 1'b0;
    repeat($random() % 'd3) @(negedge clk);
    in_valid = 1'b1; in_digit = ch_d[0];
    @(negedge clk);
    in_digit = ch_d[1];
    @(negedge clk);
    in_digit = ch_d[2];
    @(negedge clk);
    in_digit = ch_d[3];
    @(negedge clk);
    in_valid = 1'b0; in_digit = 'dx;
    golden_passwd = ch_d;
end
endtask
```

Full Code ↓



Coding Style — Parallel Check

- Create independent initial-block or fork-join to perform check in background

```
initial begin
  wait(rst_n === 1'b0);
  wait(rst_n === 1'b1);
  forever begin
    @(negedge clk); #(CYCLE * 0.1);
    if(golden_alarm && (alarm !== 1'b1 || state !== ALARM)) begin
      fail(4);
      #(CYCLE * 5);
      $fclose(f_in);
      $finish;
    end
  end
end
```



```
@(negedge clk);
in_valid = 1'b1;
in_digit = in_d[0];
check_fail_4();

@(negedge clk);
in_digit = in_d[1];
check_fail_4();

@(negedge clk);
in_digit = in_d[2];
check_fail_4();

@(negedge clk);
in_digit = 4'bx;
enter = 1'b1;
check_fail_4();

@(negedge clk);
in_valid = 1'b0;
enter = 1'b0;
check_fail_4();
```



Coding Style — force/release

- Override the value of any variable or net
- Use cases
 - Keep clock unchanged when reset
 - Error injection
 - Bypassing initialization
 - ...

